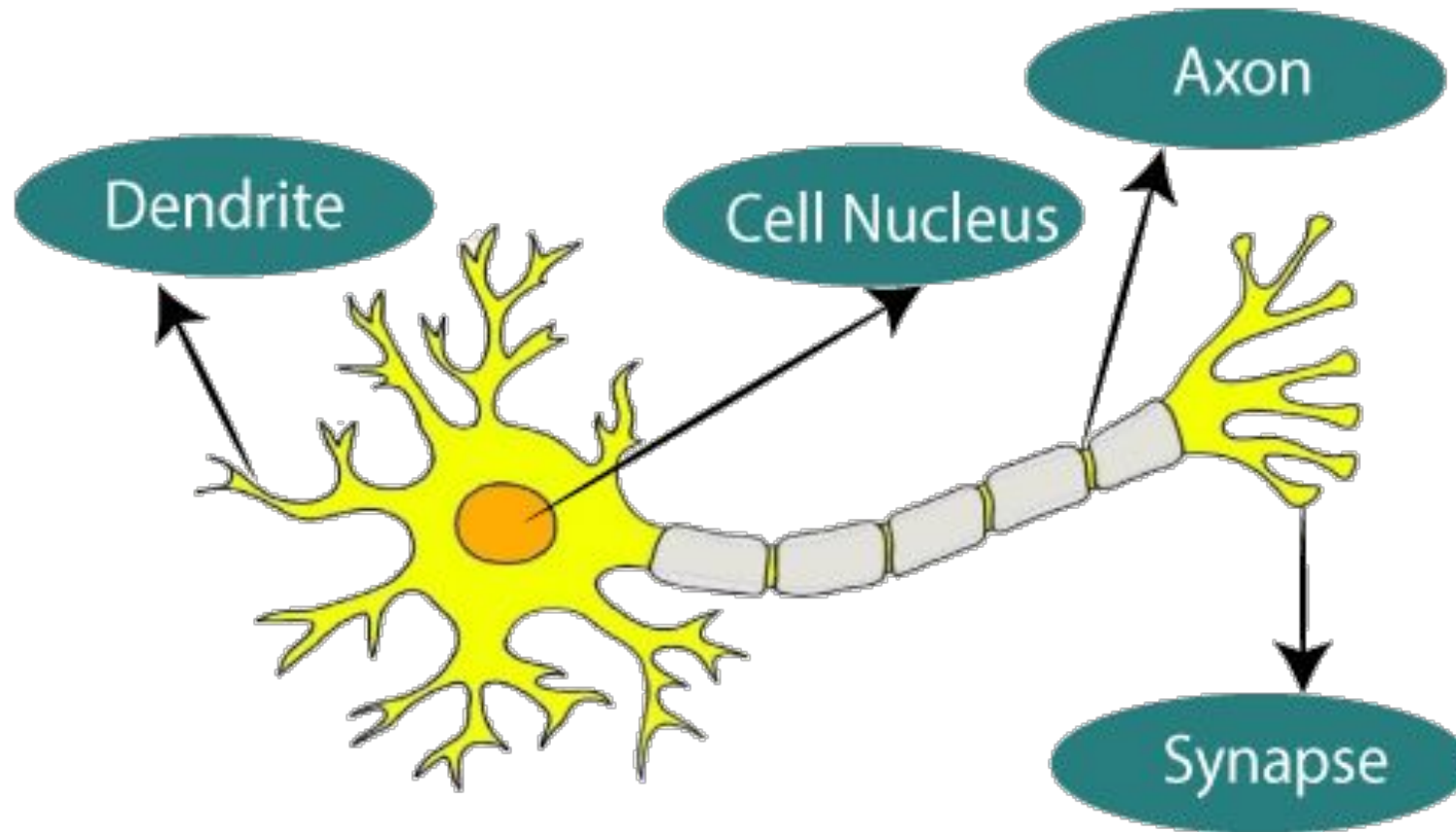
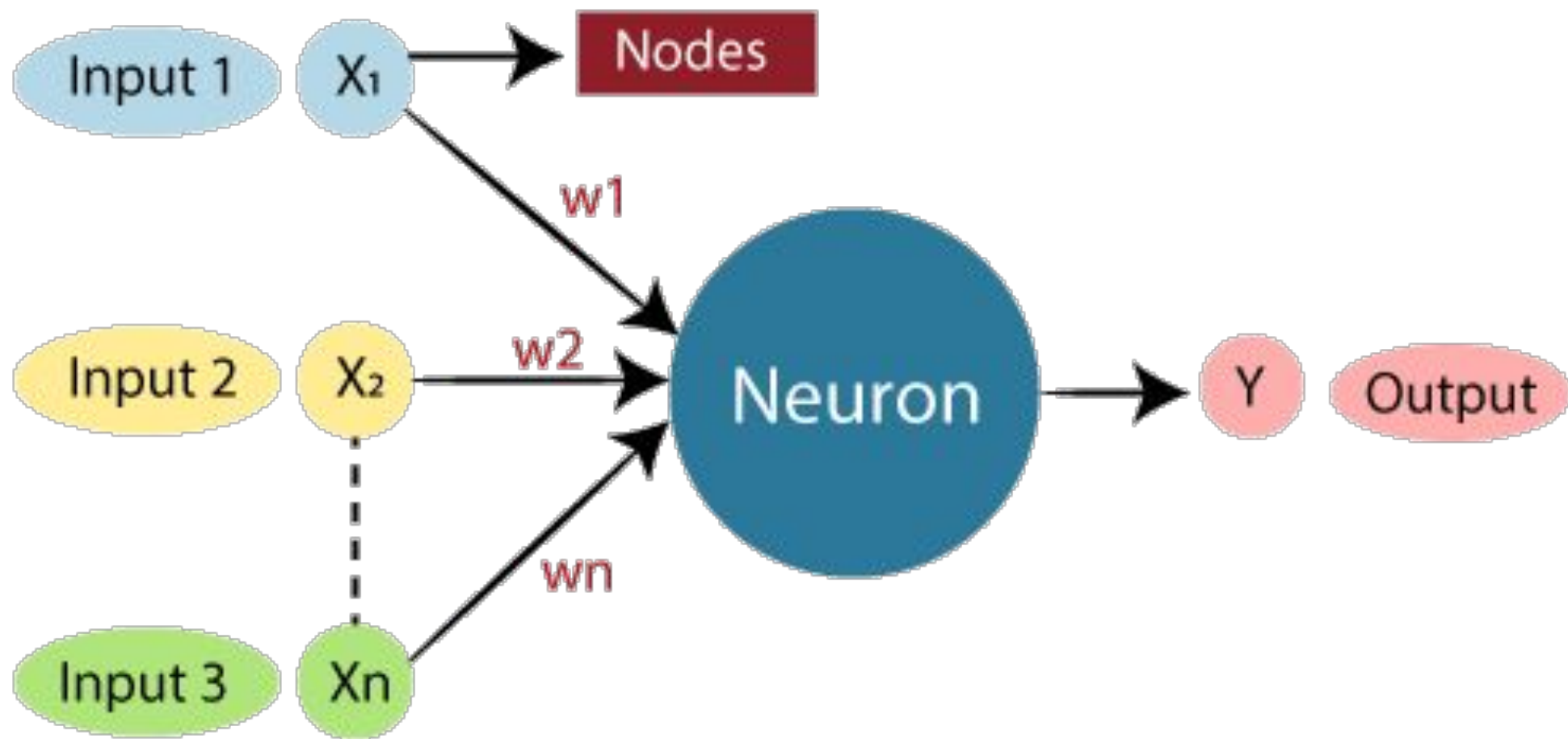


NEURAL NETWORKS

The term "**Artificial Neural Network**" is derived from Biological neural networks that develop the structure of a human brain. Similar to the human brain that has neurons interconnected to one another, artificial neural networks also have neurons that are interconnected to one another in various layers of the networks. These neurons are known as nodes.





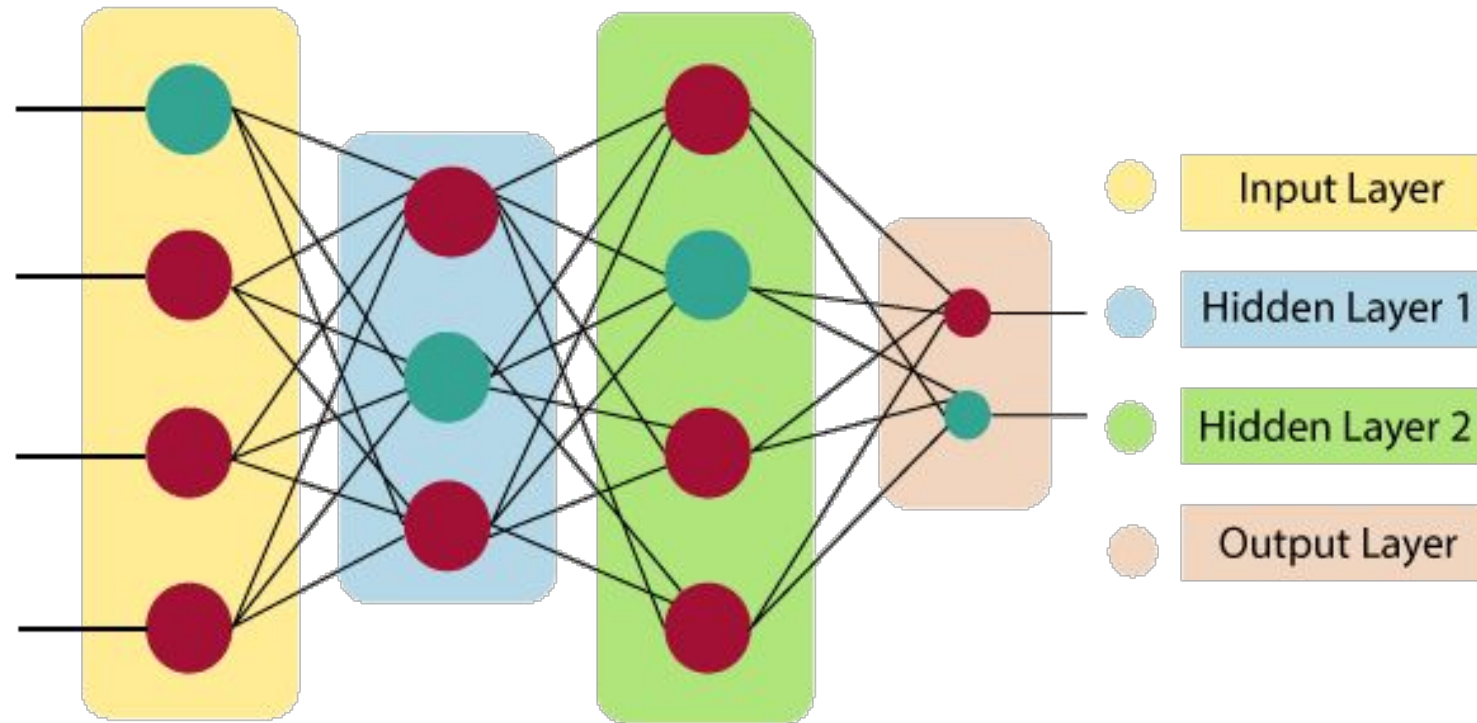
Relationship between Biological neural network and artificial neural network:

Biological Neural Network	Artificial Neural Network
Dendrites	Inputs
Cell nucleus	Nodes
Synapse	Weights
Axon	Output

The architecture of an artificial neural network:

To understand the concept of the architecture of an artificial neural network, we have to understand what a neural network consists of. In order to define a neural network that consists of a large number of artificial neurons, which are termed units arranged in a sequence of layers. Lets us look at various types of layers available in an artificial neural network.

Artificial Neural Network primarily consists of three layers:



Input Layer:

As the name suggests, it accepts inputs in several different formats provided by the programmer.

Hidden Layer:

The hidden layer presents in-between input and output layers. It performs all the calculations to find hidden features and patterns.

Output Layer:

The input goes through a series of transformations using the hidden layer, which finally results in output that is conveyed using this layer.

The artificial neural network takes input and computes the weighted sum of the inputs and includes a bias. This computation is represented in the form of a transfer function.

$$\sum_{i=1}^n W_i * X_i + b$$

It determines weighted total is passed as an input to an activation function to produce the output. Activation functions choose whether a node should fire or not. Only those who are fired make it to the output layer. There are distinctive activation functions available that can be applied upon the sort of task we are performing.

Advantages of Artificial Neural Network (ANN)

Parallel processing capability:

Artificial neural networks have a numerical value that can perform more than one task simultaneously.

Storing data on the entire network:

Data that is used in traditional programming is stored on the whole network, not on a database. The disappearance of a couple of pieces of data in one place doesn't prevent the network from working.

Capability to work with incomplete knowledge:

After ANN training, the information may produce output even with inadequate data. The loss of performance here relies upon the significance of missing data.

Having a memory distribution:

For ANN is to be able to adapt, it is important to determine the examples and to encourage the network according to the desired output by demonstrating these examples to the network. The succession of the network is directly proportional to the chosen instances, and if the event can't appear to the network in all its aspects, it can produce false output.

Having fault tolerance:

Extortion of one or more cells of ANN does not prohibit it from generating output, and this feature makes the network fault-tolerance.

Disadvantages of Artificial Neural Network:

Assurance of proper network structure:

There is no particular guideline for determining the structure of artificial neural networks. The appropriate network structure is accomplished through experience, trial, and error.

Unrecognized behavior of the network:

It is the most significant issue of ANN. When ANN produces a testing solution, it does not provide insight concerning why and how. It decreases trust in the network.

Hardware dependence:

Artificial neural networks need processors with parallel processing power, as per their structure. Therefore, the realization of the equipment is dependent.

Difficulty of showing the issue to the network:

ANNs can work with numerical data. Problems must be converted into numerical values before being introduced to ANN. The presentation mechanism to be resolved here will directly impact the performance of the network. It relies on the user's abilities.

The duration of the network is unknown:

The network is reduced to a specific value of the error, and this value does not give us optimum results.

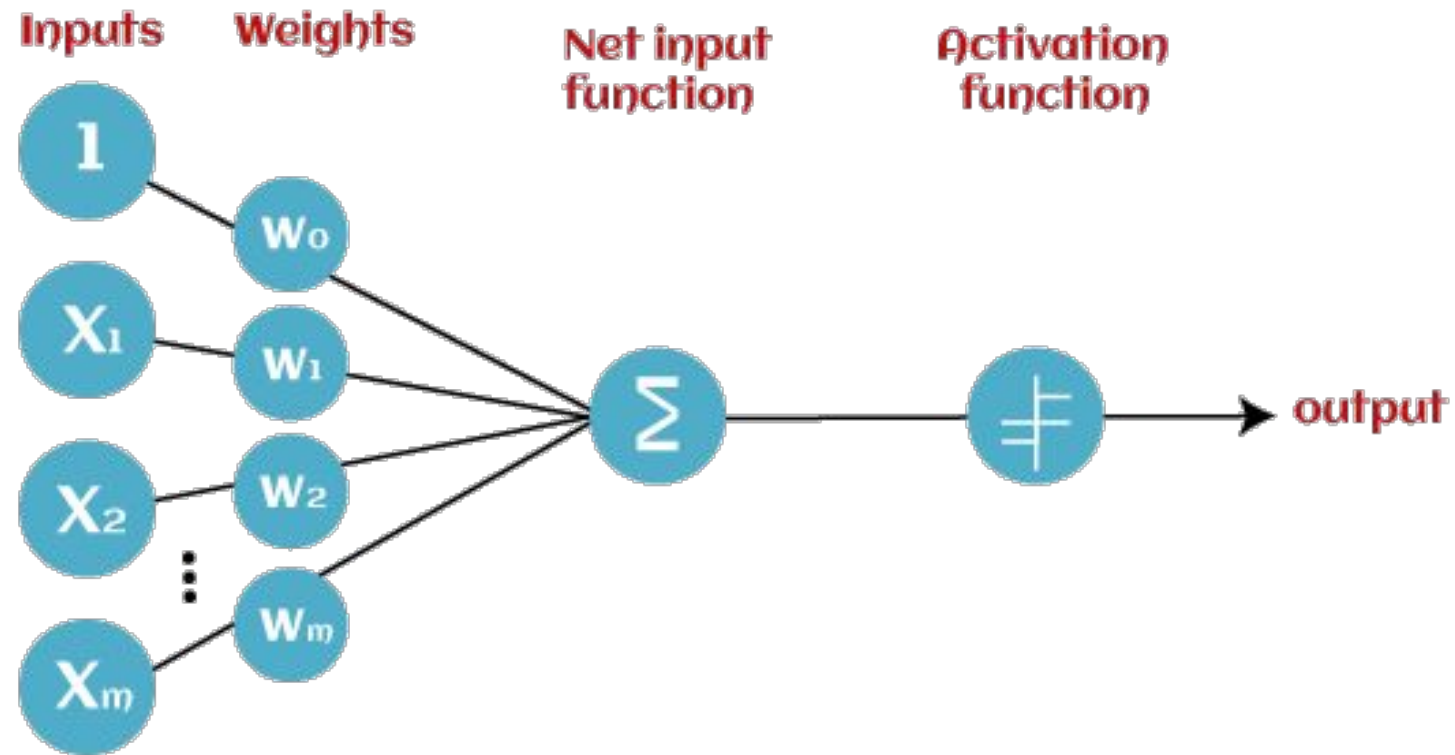
Perceptron in Machine Learning

Perceptron is Machine Learning algorithm for supervised learning of various binary classification tasks. Further, *Perceptron is also understood as an Artificial Neuron or neural network unit that helps to detect certain input data computations in business intelligence.*

Perceptron model is also treated as one of the best and simplest types of Artificial Neural networks. However, it is a supervised learning algorithm of binary classifiers. Hence, we can consider it as a single-layer neural network with four main parameters, i.e., **input values, weights and Bias, net sum, and an activation function.**

Basic Components of Perceptron

Mr. Frank Rosenblatt invented the perceptron model as a binary classifier which contains three main components. These are as follows:



- Input Nodes or Input Layer:**

This is the primary component of Perceptron which accepts the initial data into the system for further processing. Each input node contains a real numerical value.

- Wight and Bias:**

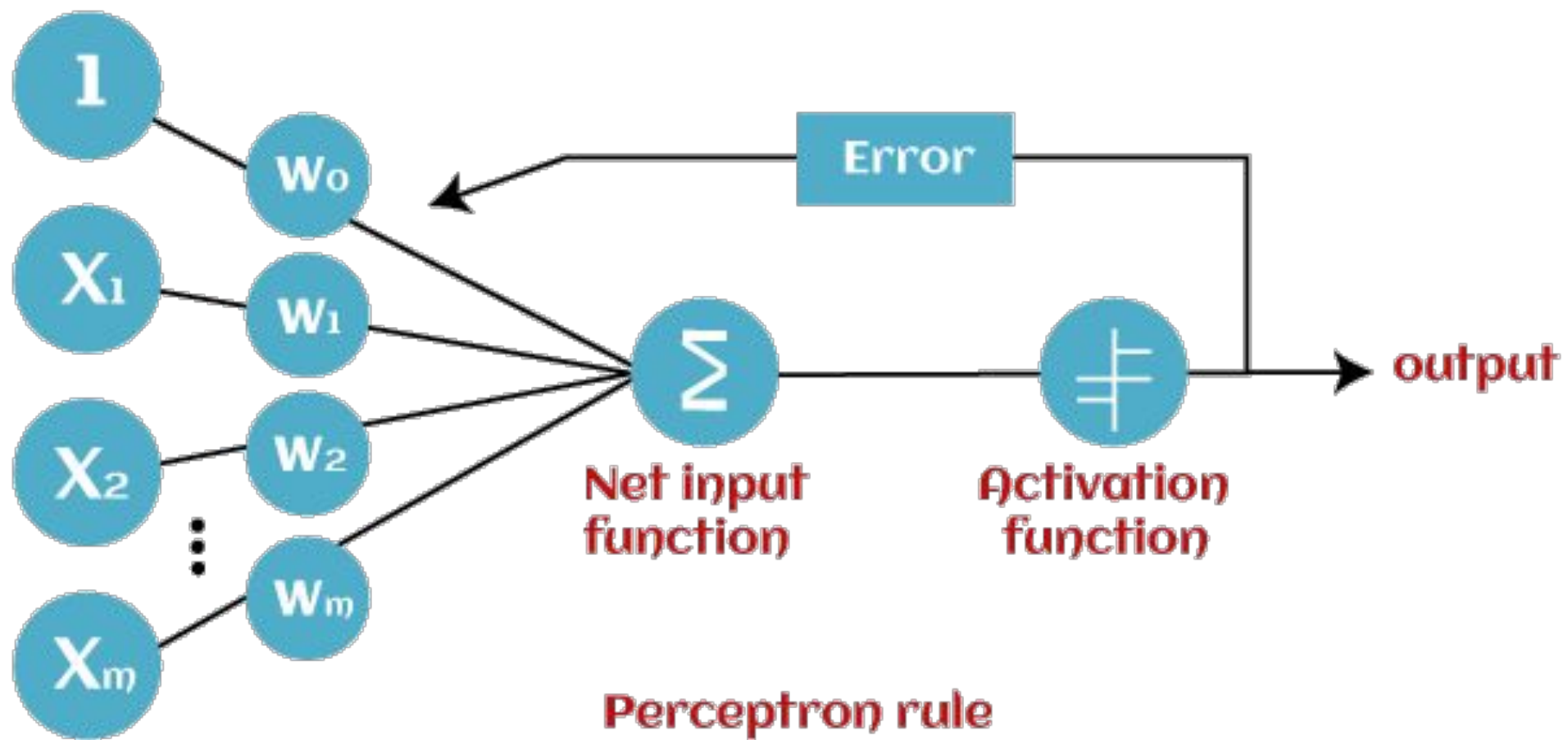
Weight parameter represents the strength of the connection between units. This is another most important parameter of Perceptron components. Weight is directly proportional to the strength of the associated input neuron in deciding the output. Further, Bias can be considered as the line of intercept in a linear equation.

- Activation Function:**

These are the final and important components that help to determine whether the neuron will fire or not. Activation Function can be considered primarily as a step function.

How does Perceptron work?

- In Machine Learning, Perceptron is considered as a single-layer neural network that consists of four main parameters named input values (Input nodes), weights and Bias, net sum, and an activation function. The perceptron model begins with the multiplication of all input values and their weights, then adds these values together to create the weighted sum. Then this weighted sum is applied to the activation function 'f' to obtain the desired output. This activation function is also known as the **step function** and is represented by 'f'.



This step function or Activation function plays a vital role in ensuring that output is mapped between required values (0,1) or (-1,1). It is important to note that the weight of input is indicative of the strength of a node. Similarly, an input's bias value gives the ability to shift the activation function curve up or down.

In general terms, these weights normally represent the strength of the interconnection between neurons inside the artificial neural network. All the weighted inputs are summarized inside the computing unit.

If the weighted sum is equal to zero, then bias is added to make the output non-zero or something else to scale up to the system's response. Bias has the same input, and weight equals to 1. Here the total of weighted inputs can be in the range of 0 to positive infinity. Here, to keep the response in the limits of the desired value, a certain maximum value is benchmarked, and the total of weighted inputs is passed through the activation function.

Perceptron model works in two important steps as follows:

Step-1

In the first step first, multiply all input values with corresponding weight values and then add them to determine the weighted sum. Mathematically, we can calculate the weighted sum as follows:

$$\sum w_i * x_i = x_1 * w_1 + x_2 * w_2 + \dots w_n * x_n$$

Add a special term called **bias 'b'** to this weighted sum to improve the model's performance.

$$\sum w_i * x_i + b$$

Step-2

In the second step, an activation function is applied with the above-mentioned weighted sum, which gives us output either in binary form or a continuous value as follows:

$$Y = f(\sum w_i * x_i + b)$$

Types of Perceptron Models

Based on the layers, Perceptron models are divided into two types. These are as follows:

1. Single-layer Perceptron Model
2. Multi-layer Perceptron model

Single Layer Perceptron Model:

A single-layered perceptron model consists **feed-forward network** and also includes a threshold transfer function inside the model. The main objective of the single-layer perceptron model is to analyze the linearly separable objects with binary outcomes.

In a single layer perceptron model, its algorithms do not contain recorded data, so it begins with inconstantly allocated input for weight parameters. Further, it sums up all inputs (weight). After adding all inputs, if the total sum of all inputs is more than a pre-determined value, the model gets activated and shows the output value as +1.

If the outcome is same as pre-determined or threshold value, then the performance of this model is stated as satisfied, and weight demand does not change. However, this model consists of a few discrepancies triggered when multiple weight inputs values are fed into the model. Hence, to find desired output and minimize errors, some changes should be necessary for the weights input.

Multi-Layered Perceptron Model:

Like a single-layer perceptron model, a multi-layer perceptron model also has the same model structure but has a greater number of hidden layers.

The multi-layer perceptron model is also known as the **Backpropagation algorithm**, which executes in two stages as follows:

- Forward Stage:** Activation functions start from the input layer in the forward stage and terminate on the output layer.
- Backward Stage:** In the backward stage, weight and bias values are modified as per the model's requirement. In this stage, the error between actual output and demanded originated backward on the output layer and ended on the input layer.

Hence, a multi-layered perceptron model has considered as multiple artificial neural networks having various layers in which activation function does not remain linear, similar to a single layer perceptron model. Instead of linear, activation function can be executed as sigmoid, TanH, ReLU, etc., for deployment.

A multi-layer perceptron model has greater processing power and can process linear and non-linear patterns. Further, it can also implement logic gates such as AND, OR, XOR, NAND, NOT, XNOR, NOR.

Feed forward neural network

Feed forward neural networks are artificial neural networks in which nodes do not form loops. This type of neural network is also known as a multi-layer neural network as all information is only passed forward.

During data flow, input nodes receive data, which travel through hidden layers, and exit output nodes. No links exist in the network that could get used to by sending information back from the output node.

A feed forward neural network approximates functions in the following way:

- An algorithm calculates classifiers by using the formula $y = f^*(x)$.
- Input x is therefore assigned to category y .
- According to the feed forward model, $y = f(x; \theta)$. This value determines the closest approximation of the function.

When the feed forward neural network gets simplified, it can appear as a single layer perceptron.

This model multiplies inputs with weights as they enter the layer. Afterward, the weighted input values get added together to get the sum. As long as the sum of the values rises above a certain threshold, set at zero, the output value is usually 1, while if it falls below the threshold, it is usually -1.

As a feed forward neural network model, the single-layer perceptron often gets used for classification. Machine learning can also get integrated into single-layer perceptrons. Through training, neural networks can adjust their weights based on a property called the delta rule, which helps them compare their outputs with the intended values.

As a result of training and learning, gradient descent occurs. Similarly, multi-layered perceptrons update their weights. But, this process gets known as back-propagation. If this is the case, the network's hidden layers will get adjusted according to the output values produced by the final layer.

Layers of feed forward neural network

- Input layer:

The neurons of this layer receive input and pass it on to the other layers of the network. Feature or attribute numbers in the dataset must match the number of neurons in the input layer.

- Output layer:

According to the type of model getting built, this layer represents the forecasted feature.

- Hidden layer:

Input and output layers get separated by hidden layers. Depending on the type of model, there may be several hidden layers.

There are several neurons in hidden layers that transform the input before actually transferring it to the next layer.

This network gets constantly updated with weights in order to make it easier to predict.

- Neuron weights:

Neurons get connected by a weight, which measures their strength or magnitude. Similar to linear regression coefficients, input weights can also get compared.

Weight is normally between 0 and 1, with a value between 0 and 1.

- Neurons:

Artificial neurons get used in feed forward networks, which later get adapted from biological neurons. A neural network consists of artificial neurons.

Neurons function in two ways: first, they create weighted input sums, and second, they activate the sums to make them normal.

Activation functions can either be linear or nonlinear. Neurons have weights based on their inputs. During the learning phase, the network studies these weights.

- Activation Function:

Neurons are responsible for making decisions in this area.

According to the activation function, the neurons determine whether to make a linear or nonlinear decision. Since it passes through so many layers, it prevents the cascading effect from increasing neuron outputs.

An activation function can be classified into three major categories: sigmoid, Tanh, and Rectified Linear Unit (ReLu).

- Sigmoid:

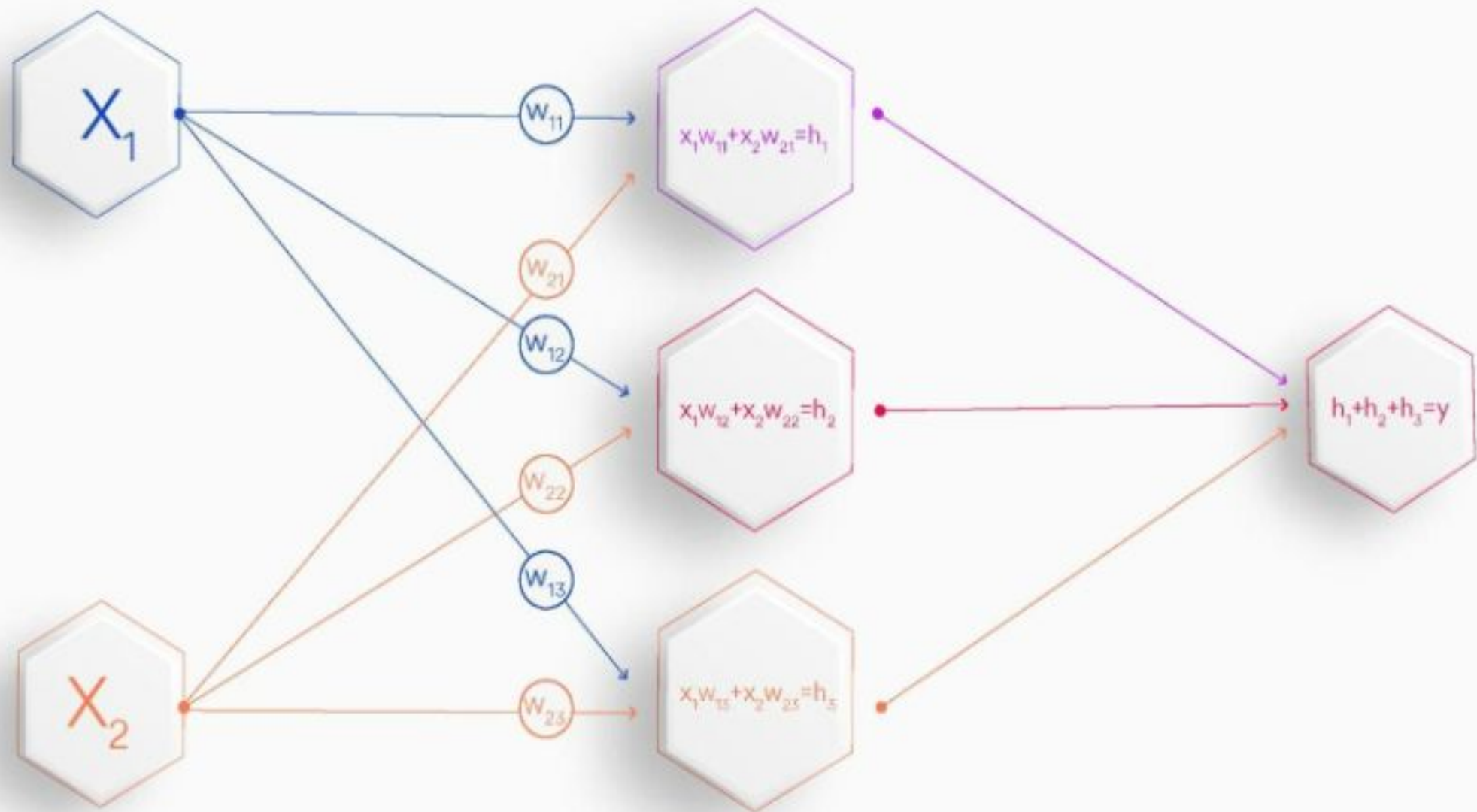
Input values between 0 and 1 get mapped to the output values.

- Tanh:

A value between -1 and 1 gets mapped to the input values.

- Rectified linear Unit:

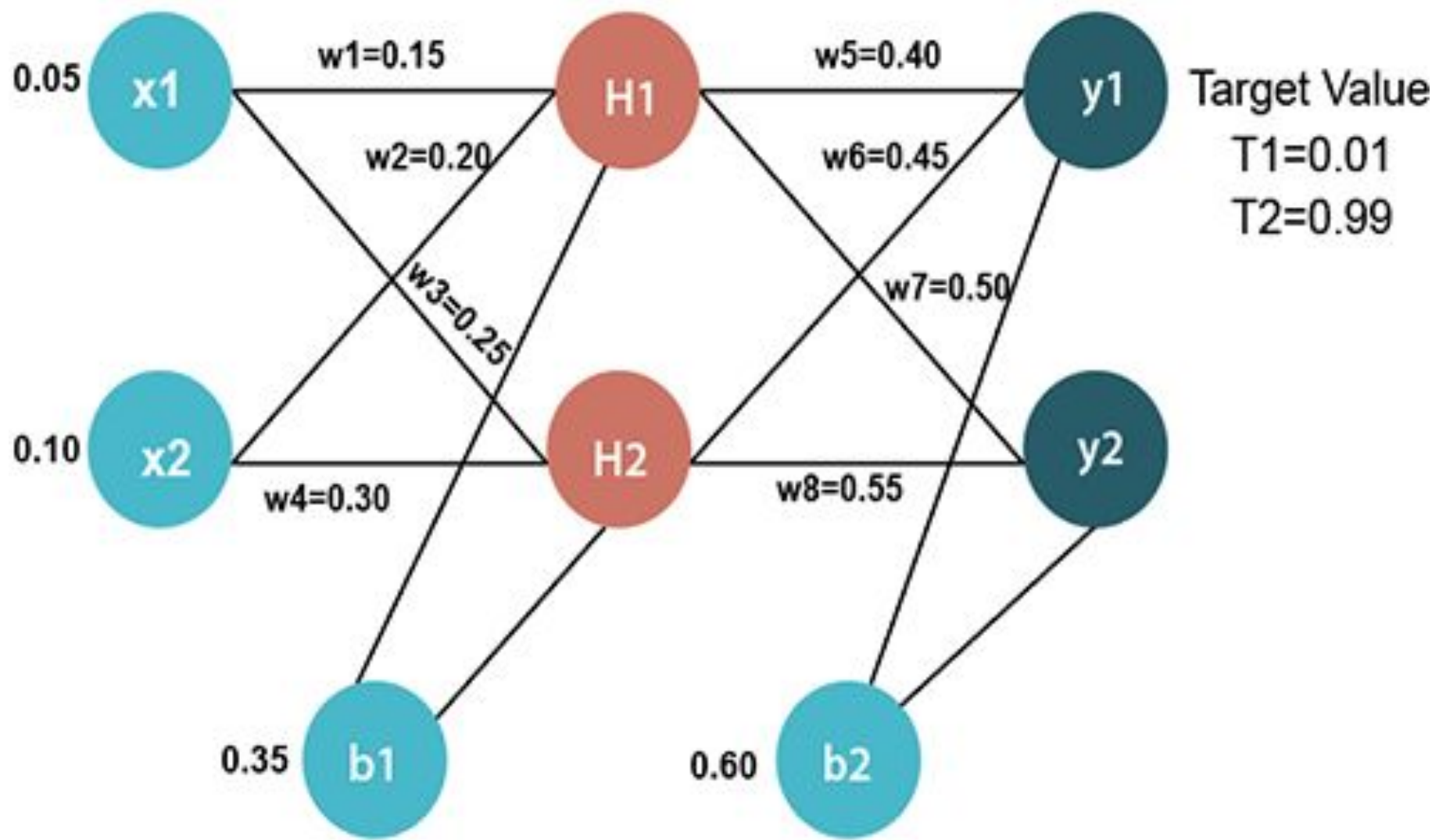
Only positive values are allowed to flow through this function. Negative values get mapped to 0.



In a matrix format, it looks as follows:

$$\begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} = \begin{bmatrix} x_1 w_{11} + x_2 w_{21} \\ x_1 w_{12} + x_2 w_{22} \\ x_1 w_{13} + x_2 w_{23} \end{bmatrix}' = \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix}'$$

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix}' \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = h_1 + h_2 + h_3 = y$$



Input values

$X_1=0.05$

$X_2=0.10$

Initial weight

$W_1=0.15$ $w_5=0.40$

$W_2=0.20$ $w_6=0.45$

$W_3=0.25$ $w_7=0.50$

$W_4=0.30$ $w_8=0.55$

Bias Values

$b_1=0.35$ $b_2=0.60$

Target Values

$T_1=0.01$

$T_2=0.99$

Forward Pass

To find the value of H1 we first multiply the input value from the weights as

$$H1 = x_1 \times w_1 + x_2 \times w_2 + b_1$$

$$H1 = 0.05 \times 0.15 + 0.10 \times 0.20 + 0.35$$

$$\mathbf{H1 = 0.3775}$$

To calculate the final result of H1, we performed the sigmoid function as

$$H1_{\text{final}} = \frac{1}{1 + \frac{1}{e^{H1}}}$$

$$H1_{\text{final}} = \frac{1}{1 + \frac{1}{e^{0.3775}}}$$

$$\mathbf{H1_{\text{final}} = 0.593269992}$$

We will calculate the value of H2 in the same way as H1

$$H2 = x1 \times w_3 + x2 \times w_4 + b1$$

$$H2 = 0.05 \times 0.25 + 0.10 \times 0.30 + 0.35$$

$$\mathbf{H2 = 0.3925}$$

To calculate the final result of H2, we performed the sigmoid function as

$$H2_{\text{final}} = \frac{1}{1 + \frac{1}{e^{H2}}}$$

$$H2_{\text{final}} = \frac{1}{1 + \frac{1}{e^{0.3925}}}$$

$$\mathbf{H2_{\text{final}} = 0.596884378}$$

Now, we calculate the values of y1 and y2 in the same way as we calculate the H1 and H2.

To find the value of y1, we first multiply the input value i.e., the outcome of H1 and H2 from the weights as

$$y1 = H1 \times w_5 + H2 \times w_6 + b2$$

$$y1 = 0.593269992 \times 0.40 + 0.596884378 \times 0.45 + 0.60$$

$$\mathbf{y1 = 1.10590597}$$

To calculate the final result of y1 we performed the sigmoid function as

$$y1_{\text{final}} = \frac{1}{1 + \frac{1}{e^{y1}}}$$

$$y1_{\text{final}} = \frac{1}{1 + \frac{1}{e^{1.10590597}}}$$

$$\mathbf{y1_{\text{final}} = 0.75136507}$$

We will calculate the value of y_2 in the same way as y_1

$$y_2 = H_1 \times w_7 + H_2 \times w_8 + b_2$$

$$y_2 = 0.593269992 \times 0.50 + 0.596884378 \times 0.55 + 0.60$$

$$\mathbf{y_2 = 1.2249214}$$

To calculate the final result of H_1 , we performed the sigmoid function as

$$y_{2_{\text{final}}} = \frac{1}{1 + \frac{1}{e^{y_2}}}$$

$$y_{2_{\text{final}}} = \frac{1}{1 + \frac{1}{e^{1.2249214}}}$$

$$\mathbf{y_{2_{\text{final}}} = 0.772928465}$$

Our target values are 0.01 and 0.99. Our y_1 and y_2 value is not matched with our target values T_1 and T_2 .

Now, we will find the **total error**, which is simply the difference between the outputs from the target

$$E_{\text{total}} = \sum \frac{1}{2} (\text{target} - \text{output})^2$$

So, the total error is

$$\begin{aligned} &= \frac{1}{2} (t_1 - y_{1\text{final}})^2 + \frac{1}{2} (T_2 - y_{2\text{final}})^2 \\ &= \frac{1}{2} (0.01 - 0.75136507)^2 + \frac{1}{2} (0.99 - 0.772928465)^2 \\ &= 0.274811084 + 0.0235600257 \\ &\mathbf{E_{\text{total}} = 0.29837111} \end{aligned}$$

Now, we will backpropagate this error to update the weights using a backward pass.

Backpropagation network

For a single training example, **Backpropagation** algorithm calculates the gradient of the **error function**. Backpropagation can be written as a function of the neural network. Backpropagation algorithms are a set of methods used to efficiently train artificial neural networks following a gradient descent approach which exploits the chain rule.

The main features of Backpropagation are the iterative, recursive and efficient method through which it calculates the updated weight to improve the network until it is not able to perform the task for which it is being trained. Derivatives of the activation function to be known at network design time is required to Backpropagation.

Features of Backpropagation:

- 1.It is the gradient descent method as used in the case of simple perceptron network with the differentiable unit.
- 2.It is different from other networks in respect to the process by which the weights are calculated during the learning period of the network.
- 3.Training is done in the three stages :
 1. the feed-forward of input training pattern
 2. the calculation and backpropagation of the error
 3. updation of the weight

Working of Backpropagation:

Neural networks use supervised learning to generate output vectors from input vectors that the network operates on. It compares generated output to the desired output and generates an error report if the result does not match the generated output vector. Then it adjusts the weights according to the error report to get your desired output.

Backpropagation Algorithm:

Step 1: Inputs X, arrive through the preconnected path.

Step 2: The input is modeled using true weights W. Weights are usually chosen randomly.

Step 3: Calculate the output of each neuron from the input layer to the hidden layer to the output layer.

Step 4: Calculate the error in the outputs

Backpropagation Error= Actual Output – Desired Output

Step 5: From the output layer, go back to the hidden layer to adjust the weights to reduce the error.

Step 6: Repeat the process until the desired output is achieved.

Types of Backpropagation

There are two types of backpropagation networks.

- Static backpropagation:** Static backpropagation is a network designed to map static inputs for static outputs. These types of networks are capable of solving static classification problems such as OCR (Optical Character Recognition).
- Recurrent backpropagation:** Recursive backpropagation is another network used for fixed-point learning. Activation in recurrent backpropagation is feed-forward until a fixed value is reached. Static backpropagation provides an instant mapping, while recurrent backpropagation does not provide an instant mapping.

Backpropagation Step-by-Step Derivation

Given:

Inputs: $x_1=0.05$, $x_2=0.10$

Targets: $t_1=0.01$, $t_2=0.99$

Learning rate $\eta=0.5$

Activation: $\sigma(x)=1/(1+e^{-x})$

Derivative for Output Weight w5

Chain Rule

$$\partial E / \partial w_5 = (\partial E / \partial y_1)(\partial y_1 / \partial \text{neto}_1)(\partial \text{neto}_1 / \partial w_5)$$

Error derivative

$$E = 1/2(y_1 - t_1)^2$$

$$\partial E / \partial y_1 = y_1 - t_1 = 0.75136507 - 0.01 = 0.74136507$$

Sigmoid derivative

$$\partial y_1 / \partial \text{neto}_1 = y_1(1 - y_1) = 0.75136507 \times 0.24863493 = 0.186815602$$

$$\partial y_1 / \partial \text{neto}_1 = \partial(1 / (1 + e^{\text{neto}_1})) / \partial \text{neto}_1$$

$$Y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Rewrite it as:

$$Y = (1 + e^{-z})^{-1}$$

Now differentiate:

$$\frac{dY}{dz} = -1(1 + e^{-z})^{-2} \cdot \frac{d}{dz}(1 + e^{-z})$$

The derivative of $1 + e^{-z}$ is:

$$\frac{d}{dz}(1 + e^{-z}) = -e^{-z}$$

So:

$$\begin{aligned} \frac{dY}{dz} &= -(1 + e^{-z})^{-2}(-e^{-z}) \\ \frac{dY}{dz} &= \frac{e^{-z}}{(1 + e^{-z})^2} \end{aligned}$$

Convert it into sigmoid form

Now take the sigmoid expression:

$$Y = \frac{1}{1 + e^{-z}}$$

Then:

$$1 - Y = 1 - \frac{1}{1 + e^{-z}}$$

Put over common denominator:

$$1 - Y = \frac{1 + e^{-z} - 1}{1 + e^{-z}} = \frac{e^{-z}}{1 + e^{-z}}$$

Now multiply Y and $1 - Y$:

$$Y(1 - Y) = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}$$

$$Y(1 - Y) = \frac{e^{-z}}{(1 + e^{-z})^2}$$

That is exactly the derivative we found above. So:

$$\frac{dY}{dz} = Y(1 - Y)$$

Net derivative

$$\text{Neto1} = h1w5 + h2w6 + b2 = h1$$

$$\partial \text{neto1} / \partial w5 = h1 = 0.593269992$$

Gradient

$$\partial E / \partial w5 = (\partial E / \partial y1)(\partial y1 / \partial \text{neto1})(\partial \text{neto1} / \partial w5)$$

$$\partial E / \partial w5 = (y1 - t1) * (y1)(1 - y1) * (h1)$$

$$\partial E / \partial w5 = 0.74136507 \times 0.186815602 \times 0.593269992 = 0.082167041$$

Weight update

$$w5(\text{new}) = 0.40 - 0.5 \times 0.082167041 = 0.35891648$$

Derivatives for Remaining Output Weights

$$\partial E / \partial w_5 = (y_1 - t_1) * (y_1)(1 - y_1) * (h_1)$$

$$w_6: \partial E / \partial w_6 = (y_1 - t_1) y_1 (1 - y_1) h_2 = 0.082667628; w_6(\text{new}) = 0.408666186$$

$$w_7: \partial E / \partial w_7 = (y_2 - t_2) y_2 (1 - y_2) h_1 = -0.02260254; w_7(\text{new}) = 0.51130127$$

$$w_8: \partial E / \partial w_8 = (y_2 - t_2) y_2 (1 - y_2) h_2 = -0.02274024; w_8(\text{new}) = 0.56137012$$

Hidden Layer Derivative (w1)

$$\partial E / \partial w_1 = (\partial E / \partial h_1) (\partial h_1 / \partial \text{neth}_1) (\partial \text{neth}_1 / \partial w_1)$$

where

$$\partial E / \partial h_1 = ((\partial E / \partial y_1) (\partial y_1 / \partial \text{neto}_1) (\partial \text{neto}_1 / \partial h_1) + (\partial E / \partial y_2) (\partial y_2 / \partial \text{neto}_2) (\partial \text{neto}_2 / \partial h_1))$$

$$\partial E / \partial h_1 = [(y_1 - t_1) y_1 (1 - y_1) w_5 + (y_2 - t_2) y_2 (1 - y_2) w_7] = 0.0363575$$

$$h_1 = 1 / (1 + e^{-\text{neth}_1})$$

$$\partial h_1 / \partial \text{neth}_1 = h_1 (1 - h_1)$$

$$\text{neth}_1 = x_1 w_1 + x_2 w_2 + b_1$$

$$\partial \text{neth}_1 / \partial w_1 = x_1$$

So,

$$\partial E / \partial w_1 = [(y_1 - t_1)y_1(1 - y_1)w_5 + (y_2 - t_2)y_2(1 - y_2)w_7] \times h_1(1 - h_1) \times x_1$$
$$= 0.000438568$$

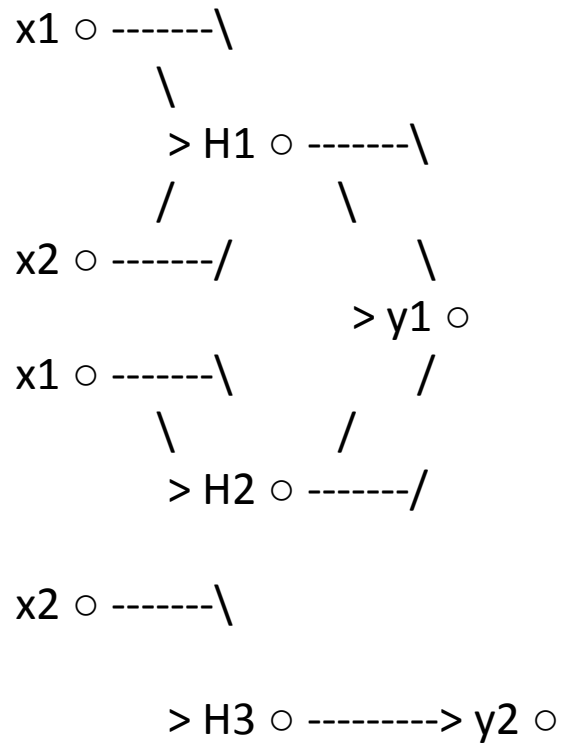
$$w_1(\text{new}) = 0.149780716$$

Weight	Gradient	Updated
w1	0.000438568	0.149780716
w5	0.082167041	0.35891648
w6	0.082667628	0.408666186
w7	-0.02260254	0.51130127
w8	-0.02274024	0.56137012

Input Layer

Hidden Layer

Output Layer



Inputs

- $x1 = 0.25$

- $x2 = 0.40$

Hidden Bias

- 0.35

Output Bias

- 0.60

Targets

- $t1 = 0.01$

- $t2 = 0.99$